

Diagrama 1 - Arquitectura de software (driver C + libstats escalar / AVX2)

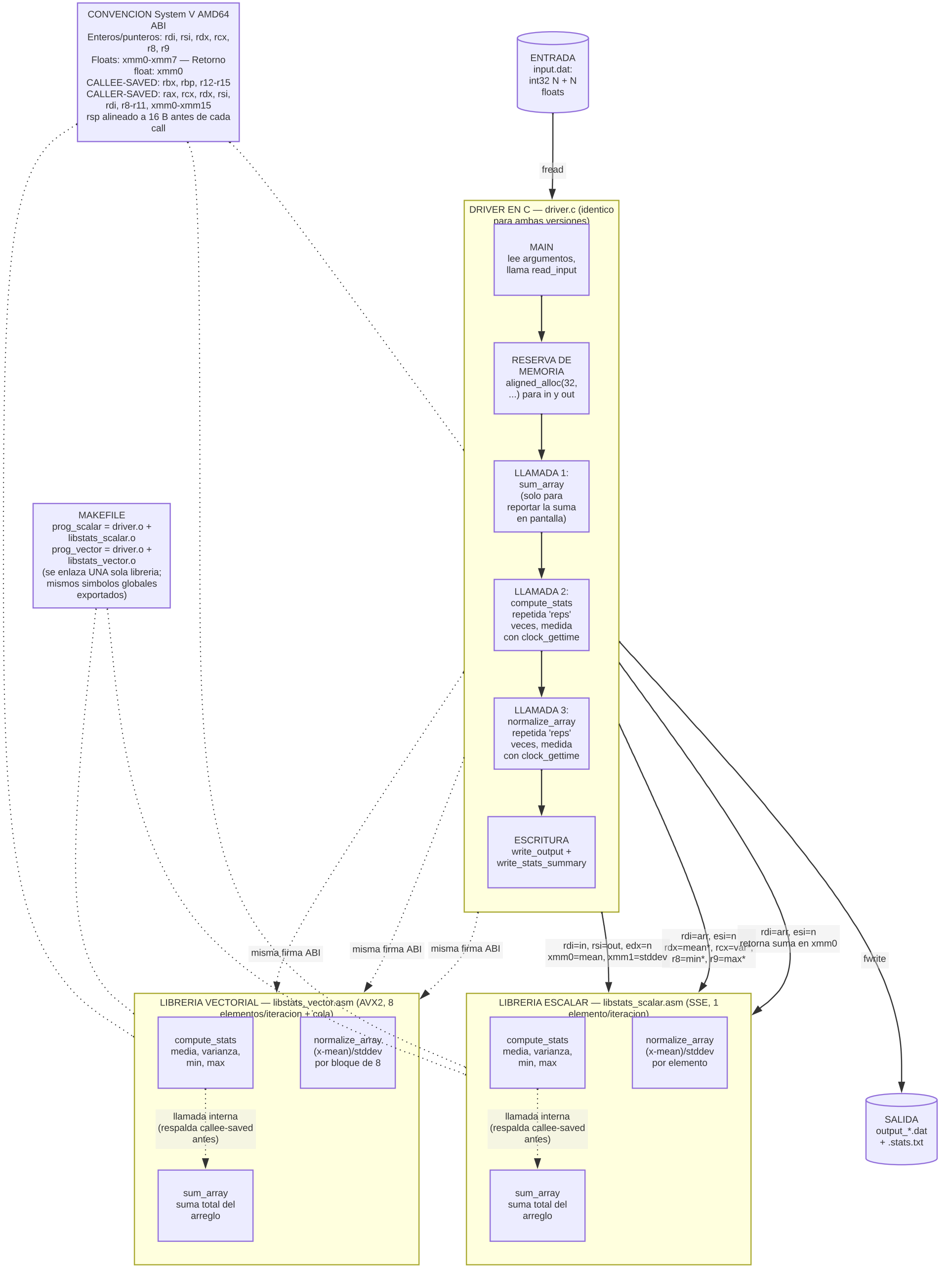


Diagrama 2 - Flujo de control del bucle principal (escalar vs AVX2)

DIFERENCIA ESTRUCTURAL
Escalar: n vueltas de 1 elemento, sin cola
Vectorial: n=8 vueltas de 8 + cola de n mod 8
Ej. n=20: 2 vueltas (16 elem.) + cola de 4

NOTA: la varianza reutiliza el patrón de sum_array (carga en double, vaddpd, reducción horizontal vhaddpd). El mínimo/máximo usa carga y reducción en float (vextractf128+vshufps), sin double. normalize_array no reduce: opera elemento a elemento sobre los 8 carriles.

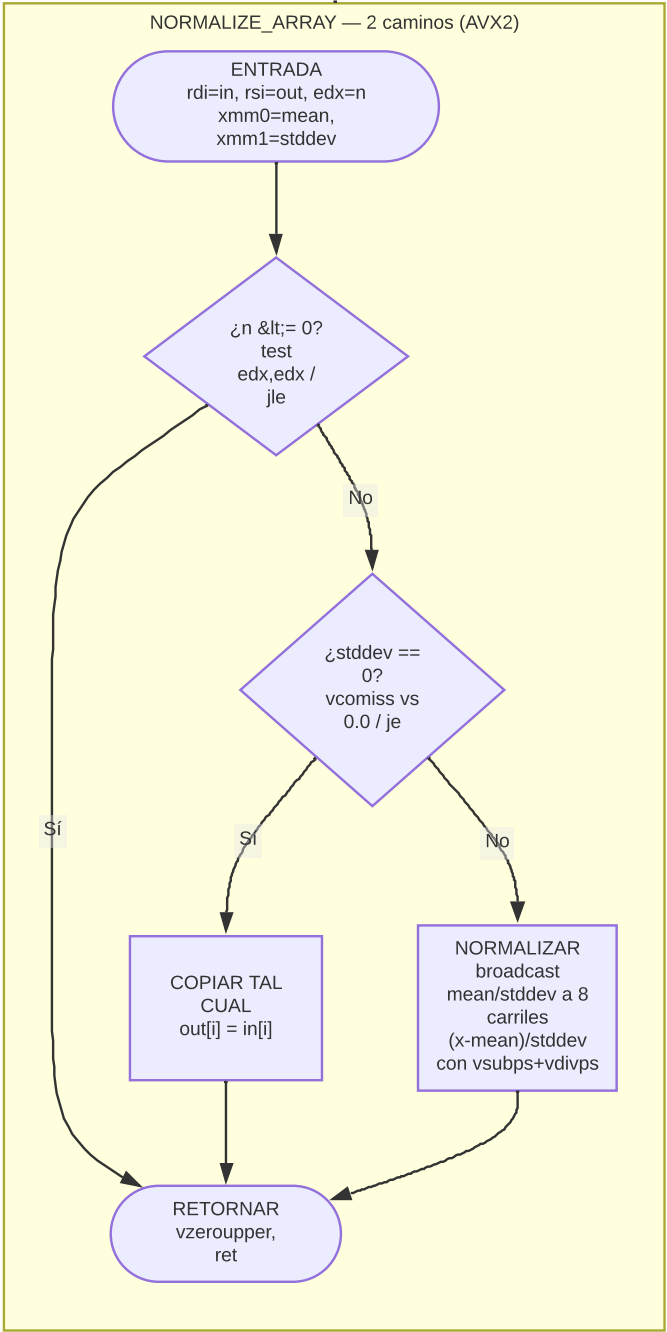
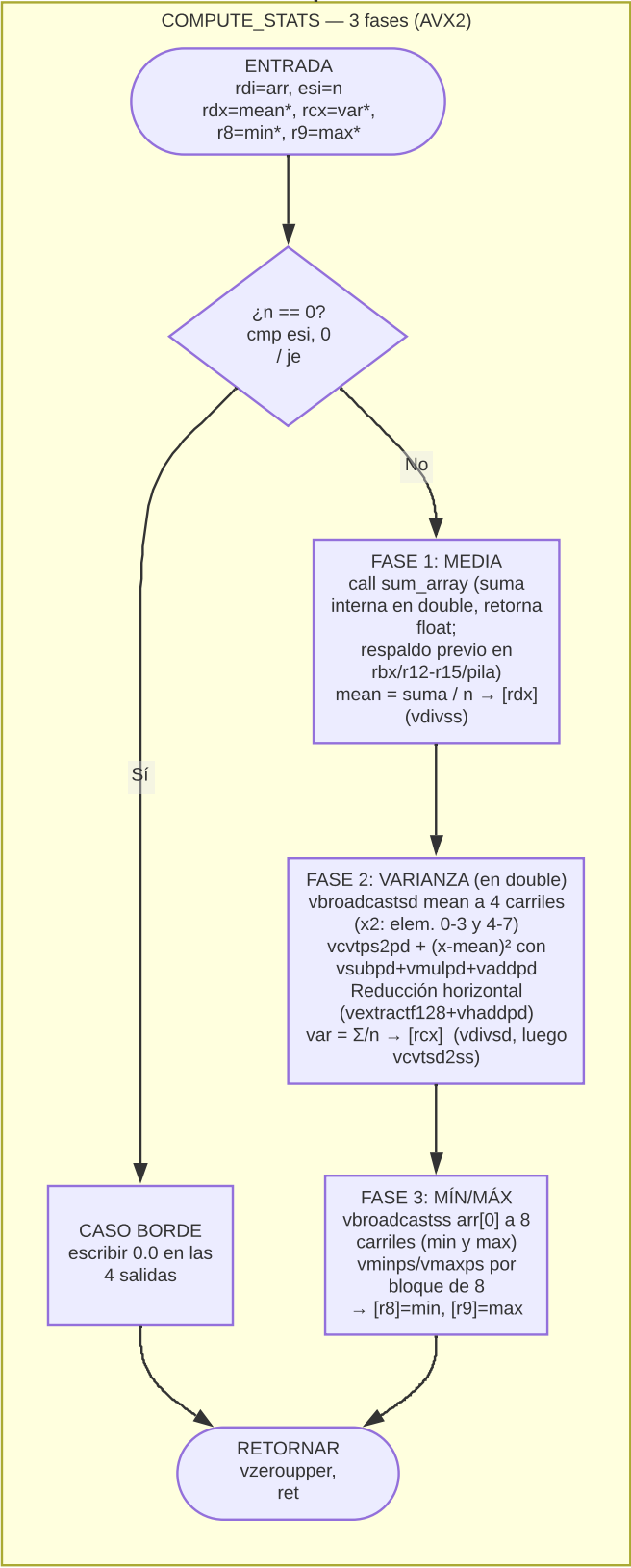
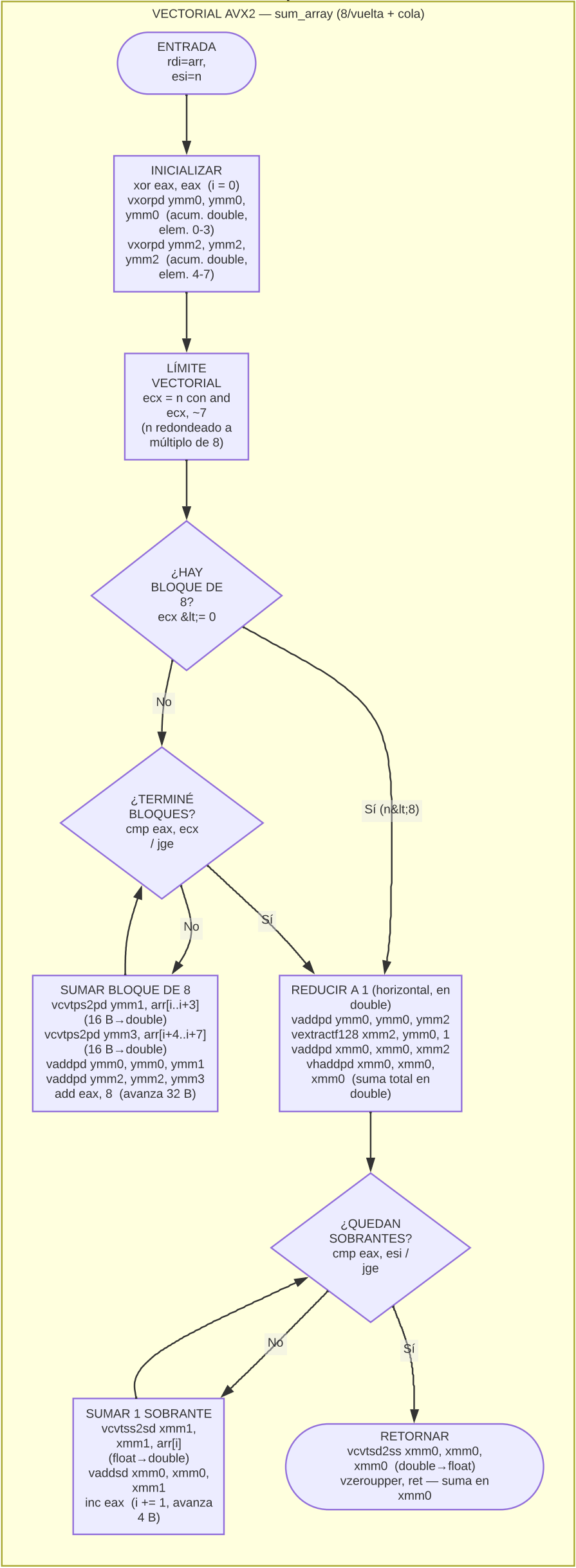
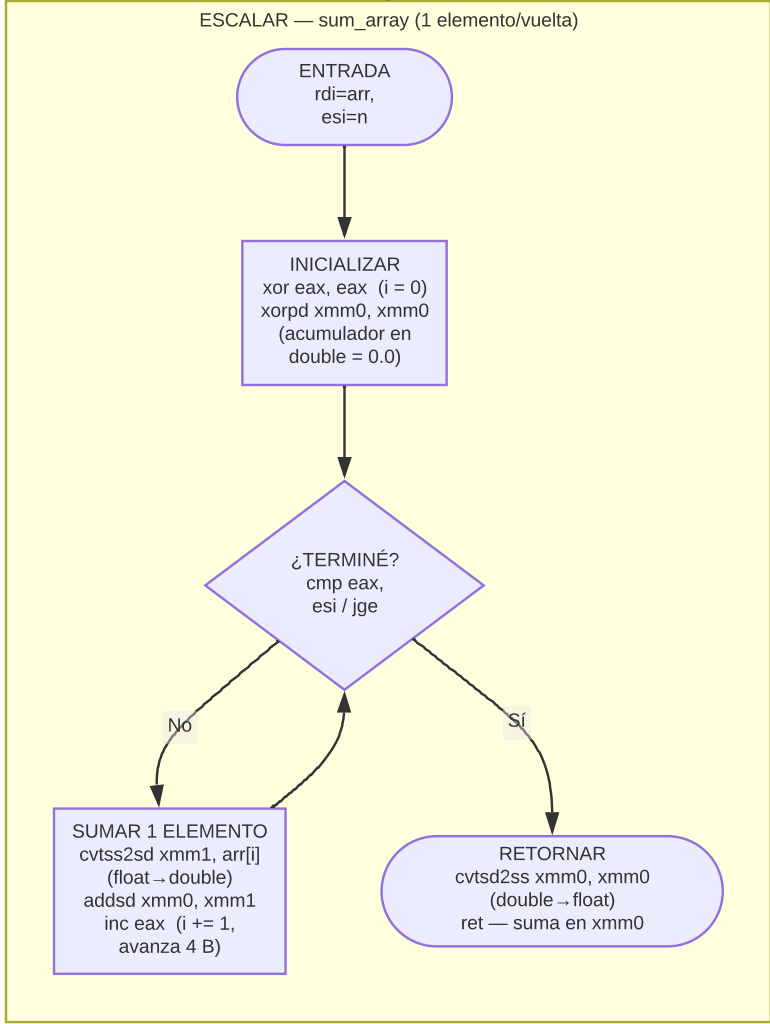


Diagrama 3 - Qué guarda cada registro en cada etapa (kernel vectorial AVX2, con la versión escalar como referencia)

ETAPA 1 - Entrada a compute_stats (si n = 0 escribe 0.0 en las 4 salidas y termina)

Registro	En palabras simples	Contenido concreto
rdi	Recibo el arreglo de entrada	puntero arr (alineado a 32 B)
esi	Recibo el tamaño	n = cantidad de floats
rdx	Recibo dónde dejar la media	puntero mean*
rcx	Recibo dónde dejar la varianza	puntero var*
r8	Recibo dónde dejar el mínimo	puntero min*
r9	Recibo dónde dejar el máximo	puntero max*

ETAPA 2 - Respaldo antes de llamar a sum_array (el call destruye los caller-saved)

Registro	En palabras simples	Contenido concreto
rbx	Guardo dónde va la media	copia de mean* (callee-saved)
r12	Guardo dónde va la varianza	copia de var*
r13	Guardo dónde va el mínimo	copia de min*
r14	Guardo dónde va el máximo	copia de max*
r15	Guardo el arreglo de entrada	copia de arr
pila	Guardo n (dos veces)	2 x push rsi: deja rsp alineado a 16 B

ETAPA 3 - Media: sum_array acumulando en double (8 elementos por vuelta)

Registro	En palabras simples	Contenido concreto
rdi / esi	Recibo arreglo y tamaño	arr y n, igual que en la ABI
eax	Llevo el índice i	0, 8, 16, ... (sube de 8 en 8 = 32 B)
ecx	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0 / ymm2	Guardo dos acumuladores en double	ymm0 = elementos 0-3, ymm2 = elementos 4-7; 4 sumas parciales
ymm1 / ymm3	Guardo el bloque convertido a double	vcvtps2pd: dos cargas de 16 B, 4 floats -> 4 doubles cada una
xmm2	Uno los dos acumuladores y guardo la mitad alta	vaddpd ymm0,ymm0,ymm2; luego vextractf128 (carriles 2-3,
xmm0	Dejo la suma total	tras vaddpd+vhaddpd, en double; sum_array la convierte a float
xmm1	Guardo n como float	vcvtssi2ss desde esi
xmm0	Calculo la media	suma (ya en float) / n, se guarda en [rdx]

ETAPA 4 - Varianza: suma de (x - media)^2 acumulada en double

Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8 (recalculado)
ymm0 / ymm4	Guardo dos acumuladores en double	ymm0 = elementos 0-3, ymm4 = elementos 4-7; sumas parciales
ymm1	Guardo la media en double, repetida en 4 carriles	vcvtss2sd desde [rdx], luego vbroadcastsd
ymm2 / ymm3	Guardo el bloque de trabajo en double	x convertido con vcvtps2pd, luego x-media, luego su
xmm2	Uno los dos acumuladores y guardo la mitad alta	vaddpd ymm0,ymm0,ymm4; luego vextractf128 (carriles 2-3,
xmm1	Guardo n convertido a double	vcvtssi2sd desde esi
xmm0	Dejo la varianza	suma de cuadrados / n en double (vdivsd)
xmm0	Guardo el resultado final	convertido a float (vcvtss2sd) y guardado en [rcx]

ETAPA 5 - Mínimo y máximo con vminps / vmaxps (8 floats por vuelta)

Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 8, 16, ... (de 8 en 8)
r10d	Guardo el límite vectorial	n redondeado hacia abajo a múltiplo de 8
ymm0	Guardo los mínimos parciales	8 carriles, inicializados con arr[0] (vbroadcastss)
ymm1	Guardo los máximos parciales	8 carriles, inicializados con arr[0] (vbroadcastss)
ymm2	Guardo el bloque actual	8 floats de arr
xmm2 / xmm3	Guardo temporales para reducir	mitad alta (vextractf128) y carriles intercambiados (vshufps)
xmm0	Dejo el mínimo final	carril 0, se guarda en [r8]
xmm1	Dejo el máximo final	carril 0, se guarda en [r9]

ETAPA 6 - normalize_array: (x - media) / desviación, 8 por vuelta + cola

Registro	En palabras simples	Contenido concreto
rdi / rsi	Recibo entrada y salida	punteros in y out (alineados a 32 B)
edx	Recibo el tamaño	n
xmm0 / xmm1	Recibo la media y la desviación	mean y stddev (luego se pisan con el broadcast)
xmm8 / xmm9	Guardo copias de media y desviación	para comparar stddev con 0 y para la cola escalar
ymm0	Guardo la media en los 8 carriles	vbroadcastss desde xmm0
ymm1	Guardo la desviación en los 8 carriles	vbroadcastss desde xmm1
ymm2	Guardo el bloque de trabajo	in, luego in - media, luego dividido = out (vmovaps)
eax / r10d	Llevo el índice y el límite vectorial	i sube de 8 en 8; límite = n redondeado a múltiplo de 8
xmm3 / xmm2	Guardo el sobrante y su resultado (cola)	xmm3 = in[i]; xmm2 = (in[i] - media) / desviación

REFERENCIA - Versión escalar (SSE/SSE2): mismos roles, un solo elemento por vuelta

Registro	En palabras simples	Contenido concreto
eax	Llevo el índice i	0, 1, 2, ... (sube de 1 en 1 = 4 B)
xmm0	Guardo el acumulador en double	una sola suma en double (sum_array y varianza); en
xmm1	Guardo el elemento convertido a double	arr[i] via cvtss2sd, o (arr[i]-media)^2 en double; en
xmm2	Guardo la media en double / el elemento a comparar	en varianza: cvtss2sd desde [rbx], leída una vez y reutilizada;
xmm8 / xmm9	Guardo media y desviación	copias usadas en normalize_array (en float)
rbx, r12-r15	Respaldo de punteros	igual que en la vectorial; la media se relee de [rbx]

Cómo leer las tablas

En las etapas 3 y 4 (media y varianza), ymm se usa como 4 doubles de 64 bits (double, mayor precisión); en las etapas 5 y 6 (mínimo/máximo y normalización), ymm se usa como 8 floats de 32 bits. xmm es la mitad baja de un ymm, o un solo valor en código escalar. Callee-saved (hay que preservarlos): rbx, rbp, r12-r15. El resto son caller-saved. Límite vectorial = n redondeado hacia abajo a múltiplo de 8; los n mod 8 sobrantes van a la cola escalar. vzeroupper antes de cada ret limpia la mitad alta de los ymm.

Diagrama 4 - Memoria: arreglos in[] y out[], alineacion y avance del puntero (ejemplo n = 20)

